



Gamepad performance optimization steps

Preliminary steps description

Company address:
Droid Technologies, LLC
Ukraine
Kiev City, str Vaclav Havel, 4,
office 422

Telephone:
+380442980190

E-mail:
support@droid-technologies.com

Revision History

Document Revision	Date of changes	Matter of Change	Note	Editor
1.00	27 May 2024	First revision		Vygoniuk Oleksandr
2.00	16 June 2024	New chapter 2	Practical results have been added and conclusion has been updated.	Vygoniuk Oleksandr
2.10	04 July 2024	Chapter 2	New filter implementation has been described.	Vygoniuk Oleksandr

PREFACE	4
1. EXPECTED PERFORMANCE INCREASING STEPS	5
1.1 Hardware work analysis	5
1.2 Software work analysis	5
2 PRACTICAL RESULTS	6
Conclusion	10

PREFACE

This document describes the expected steps to increase gamepad work performance.

Generally, this task depends on numerous factors, that is why it's very important to find the correct issue source. There can be hardware problems, or driver issues or user applications peculiarities.

For example, a possible hardware issues can be not stable gamepad keys scanning so it causes subsequent issues. This is a difficult step for analysis due to board communication via USB bus, but it should be realized as much as possible.

Another layer in the problem solution is a driver communication. It is also not so easy and depends on computer performance such as free memory or CPU cores number.

Only after this, other user applications features can be analysed In the case of custom device using.

1.EXPECTED PERFORMANCE INCREASING STEPS

1.1 Hardware work analysis

This is the most important step in the custom device testing to be sure everything is working properly from the beginning.

If it is possible, we should check data processing stages using UART terminal.

If the device has no such feature, we should try to capture USB data packages to be sure of a stable connection, since the analysed gamepad is working via USB bus.

1.2 Software work analysis

All data from the USB bus are processed in the corresponding system drivers. Since the analyzed gamepad is working as a standard HID device, there is very small probability of internal driver errors. Especially if some data is coming and we can process them. So the subject of our investigations lays in the higher layers such as user applications or overlay libraries.

Preliminary analysis of the gamepad settings applications has revealed the weakest point is data reading from the HID device. The software tries to do it permanently so it can cause other threads blocking.

The biggest problem is what to use as a reference point since the operating system divides existing resources between all applications.

To have some comparable results it would be great to test the software with some expected behaviour. For example, using gamepad emulator and timestamps.

Preliminary steps to find the performance issues are:

- a) check the ability to read HID data in an asynchronous way or using data buffering;
- b) try to read gamepad data in the separated thread with a small sleep delay (it should free some CPU resources and small delays about 1 ms is not a crucial for a human reaction);
- c) try to use the gamepad to any high performance 3D visualization with a high FPS ratio to be able a relative visual estimation of performance changes;
- d) check are the used libraries able to set virtual Xbox gamepad states in one operation to use less time. Otherwise, to add small code which update virtual Xbox gamepad states only on occurred changes;
- e) maybe it makes sense to decrease slightly the gamepad reading frequency because people can't move so quickly, but every gamepad state change can cause the game scenes updating. The last item is crucial in games with numerous objects or difficult textures, in the case if every gamepad change must update scenes every time;
- f) if the previous steps give good results, we can add part of gamepad stress test as the demo to settings GUI (for example, changing shape position or texture transparency on gamepad buttons pressing). We can save gamepad presses number and timestamps in the GPU to estimate data processing delay and detect maximal reading frequency on the computer. It is something like a self calibration to find the optimal gamepad settings.

It is clear that a human will be a slower, but it can help to find the weakest point - is it gamepad state reading or some internal game limits. This last item requires to be detailed.

2 PRACTICAL RESULTS

A practical study of the device revealed a significant level of noise on the analogue lines, which manifested itself as a sharp change in the corresponding values of the joystick positions on the X, Y axes.

To estimate the level of this noise and the resources required to overcome it, several filters were added to the program on the PC side - a simple mean filter and a median filter, and the signal level was slightly multiplied by a small factor.

Since there is currently no other evaluation tool other than user evaluation, one of the gamepad testers was used, which displayed the joystick position and deviation as a percentage of full scale.

In this way, it was easy to observe that in some positions the noise could sometimes reach a third of the scale, changes reaching 3 digits places.

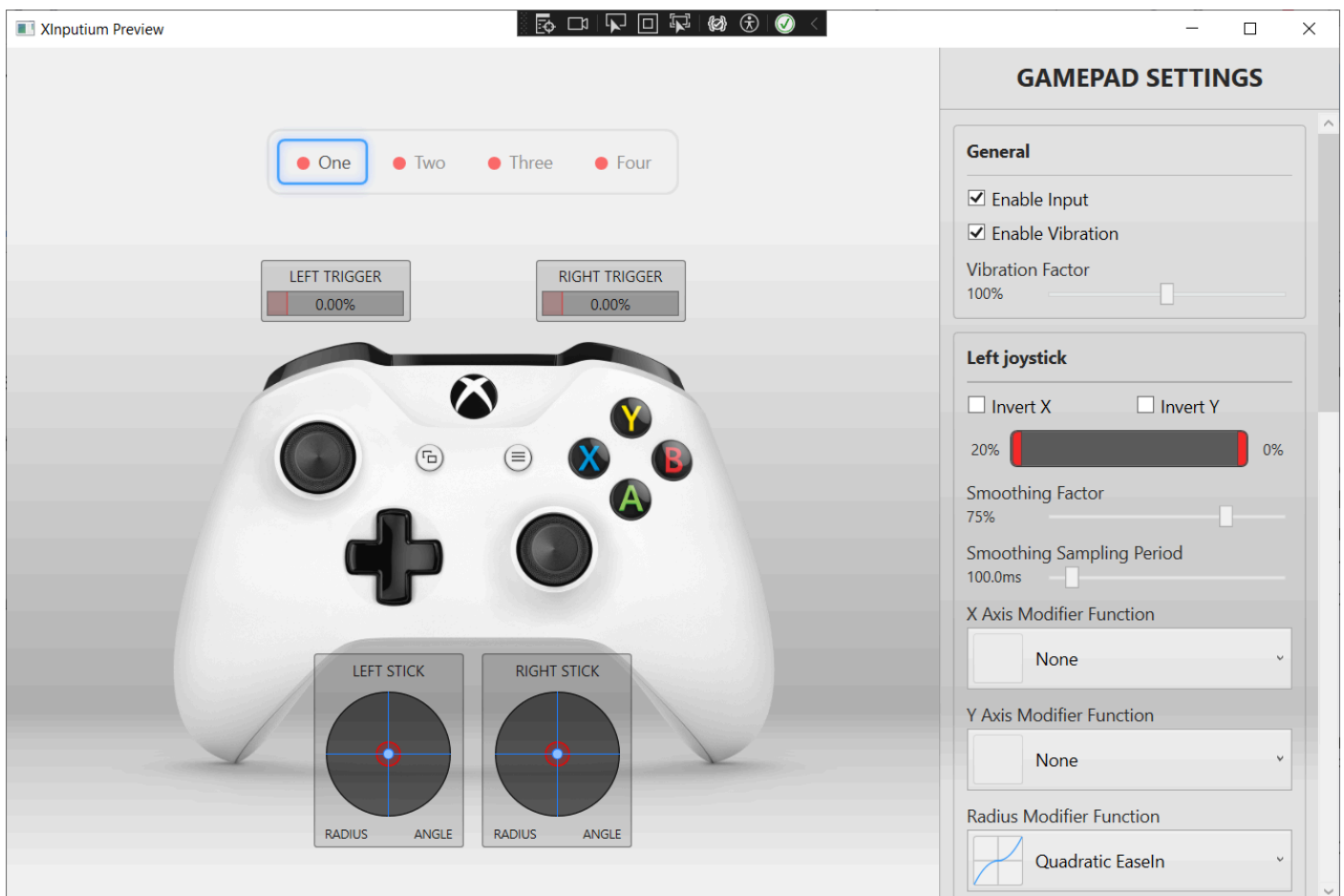


Figure 1. Used gamepad testing application

Performed tests revealed that on the PC side, the mean filter required at least 100 counts to settle to one decimal place and nearly 300 counts to settle to two decimal places. The median filter required almost 2 times less resources, but in some joystick positions the signal fluctuations were so frequent and sharp that the median value barely reached the full scale range.

In addition, each increase in the filter window introduced a significant delay to the joystick response.

In this way, there was an understanding that there is a need to use a weighted filter. If initially preference was given to the median, then further practical research showed that the weighted average filter is more suitable for time sequences in this case.

A study of the noise on the ADC lines in the board revealed significant noise reaching 35...80 mV depending on the power source, which corresponds to about 40...100 ADC counts, which is too much.

When analysing the electrical circuit of the joystick, it was found that one capacitor is missing on the digital power line and there are no capacitors on the ADC inputs.

10 nF capacitors were soldered to the analogue inputs of the ADC from the X and Y joystick lines and a 10 μ F capacitor was added to the digital supply line and a 1 μ F capacitor to the ADC supply line (see hardware review report).

Thanks to this, the required size of the filter window on the PC side was reduced by almost 12 times. But there were still significant signal fluctuations in some positions, which finally confirmed the impossibility of using the median filter.

Other optimization possibilities on the PC side that were supposed to be used in no way could eliminate the noise problem, so it was decided to make changes to the firmware of the board.

Without discussion of various details, let's say that it was necessary to adhere to the following technical parameters:

- a) the level of high-frequency noise should have decreased significantly;
- b) the filter had to be non-linear in order to respond well enough to recent quick changes and ignore noise.

The requirements are quite mutually exclusive.

It was possible to maintain the speed of the joystick response despite the addition of a filter with its own delay by increasing the ADC sampling frequency by 16 times. Through experiments, it was found that the optimal size of the filter window was also in the range of 12..16 values.

It was much more difficult to choose the optimal characteristic of the filter without delving into specialized calculations.

Experimentally, it was found that the single-chain filter was not enough, there were still quite significant fluctuations in the signal despite the presence of smoothing capacitors on the analogue lines.

Therefore, it was decided to add a simple mean filter with a small filter window size for fast filtering of high-frequency components, and to transfer the filtered value from it to the second chain with a non-linear characteristic.

The study of possible implementations quickly without the use of complex calculations led to the fact that the characteristic of the filter is approximated in different areas and had less sensitivity to the last readings, which is equivalent to a reduction in the response to very fast changes. At the same time, the amount of data in the digital filter is sufficient to form a weighted value with an appropriate stability.

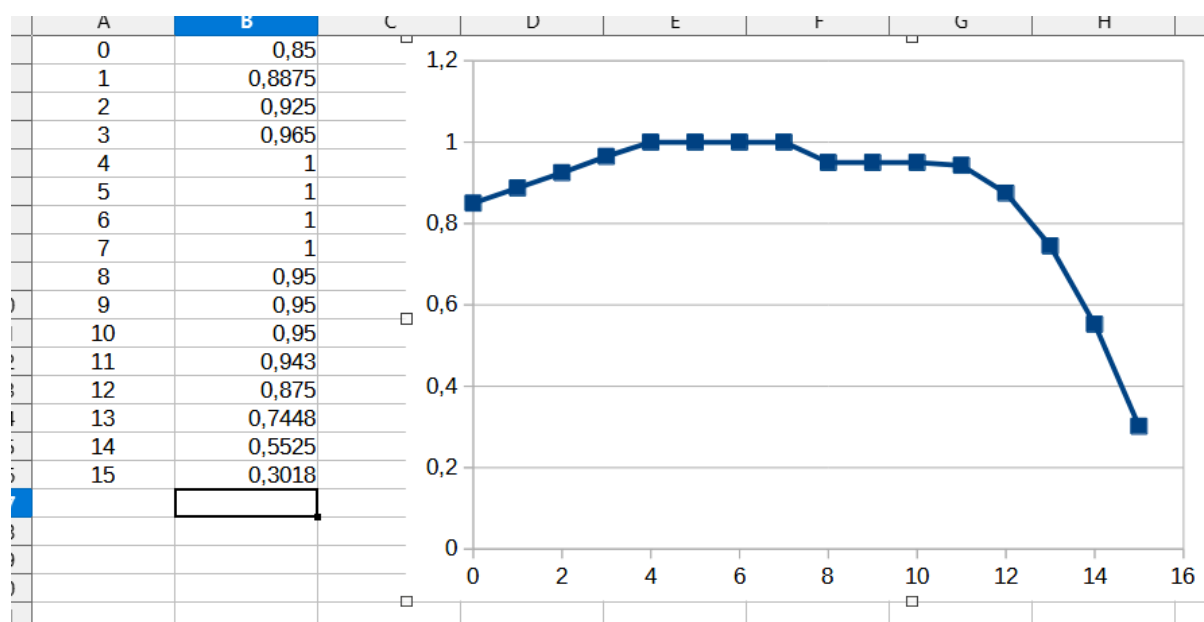


Figure 2. Approximated filter weight coefficients distribution used for the second filter chain

As a result of data processing, the level of pulsations has been significantly reduced, but there was still a slight noise that could cause minor inconveniences when using the joystick.

In order to overcome them, it was decided not to react to changes with a difference not more than 1 LSB in ADC reading, which almost completely eliminated data instability.

Additionally, an attempt was made to programmatically connect the button built into the two-axis joystick, but the experiments revealed that it has very poor quality and the activation was not always fixed.

Therefore, this part of the code is commented out and not used.

Another wish arose, based on a small experience of using the joystick under study. The joystick mechanism has a very uneven characteristic. The same joystick slope creates significantly different response changes.

Before the introduction of digital filters in the firmware, there was a noticeable instability of the mechanics, which manifested itself as a huge shaking of the signal in some positions. Currently, this problem has not disappeared, but it has significantly decreased. Even so, the data stability is almost 1..2 LSB.

At the same time, the data transfer speed in the joystick has not changed, and the joystick responds very well to rapid changes.

The last stage of optimization was the replacement of the 2-chain filter with a single-chain Savitzky-Golay filter with polynomial interpolation of the signal based on previous and subsequent readings of the data. Compared to the 2-chain filter, the filtering quality was significantly better with a smaller size of required data and a lower filter delay.

In addition, to correctly determine extreme positions of the joystick, its state was updated only after the filter window was completely filled with data.

A comparison of the performance of the selected filter can be seen in the graphs from Excel, where the red line is the input chaotic data, the green is the filtered signal for the filter with a window of 7 counts, the yellow line is 11 counts, the blue line is 19 counts.

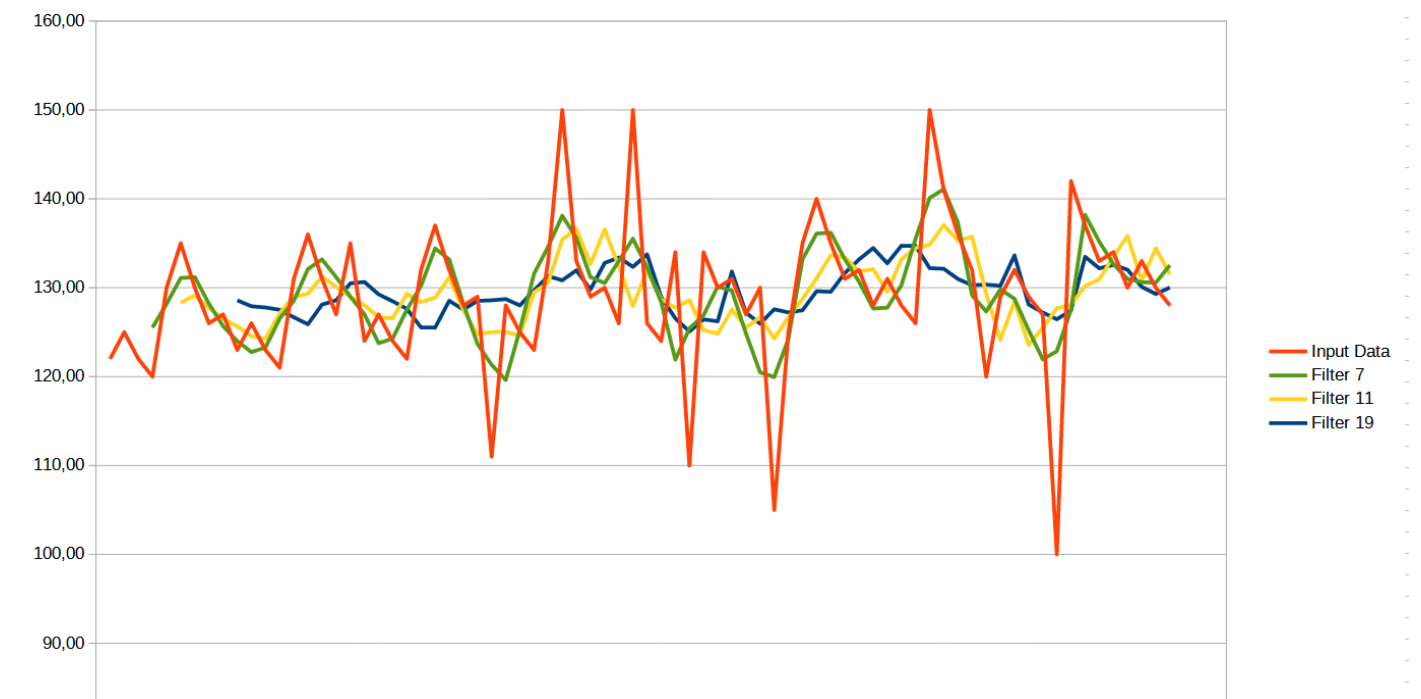


Figure 3. Estimating filter performance in Excel

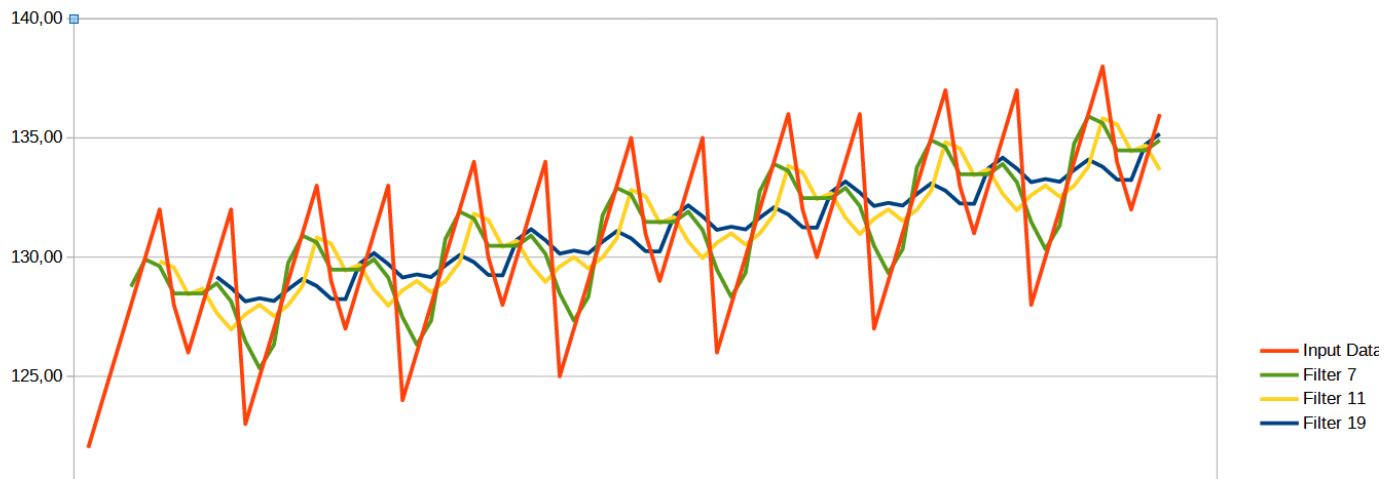


Figure 4. Estimating filter performance in Excel

It is clear that the filter is good at removing chaotic noise peaks, and when the filter window size is increased, the filtered signal has a smaller peak-to-peak range. For the joystick, a window of 19 was chosen so that the number of counts overlapped at least half of the joystick's data update period, but did not exceed this time to maintain good response to fast gamer movements.

Conclusion

All described steps reproduce the estimated plan and every step from those items could increase the gamepad performance or can help to find the performance issues sources. Some of these steps can be omitted after data reading optimization in the case it was the main problem.

Considering the combination 'gamepad-driver-user GUI' is a 'black box' with an expected behaviour, it looks sensible to estimate the performance using a reference HID emulation with predicted characteristics. It should avoid other external factors in the operating system not related to the gamepad activity.

The practical implementation of the digital filter in the firmware of the board and minor changes in the board (additional capacitors on critical lines) practically solved the existing problems, so further optimization is hardly necessary, in particular, the PC program can be left unchanged except USB thread reading optimization.

In general, the hardware, firmware and PC software are prepared professionally enough, so only minor edits had to be made in critical areas in the board and firmware.